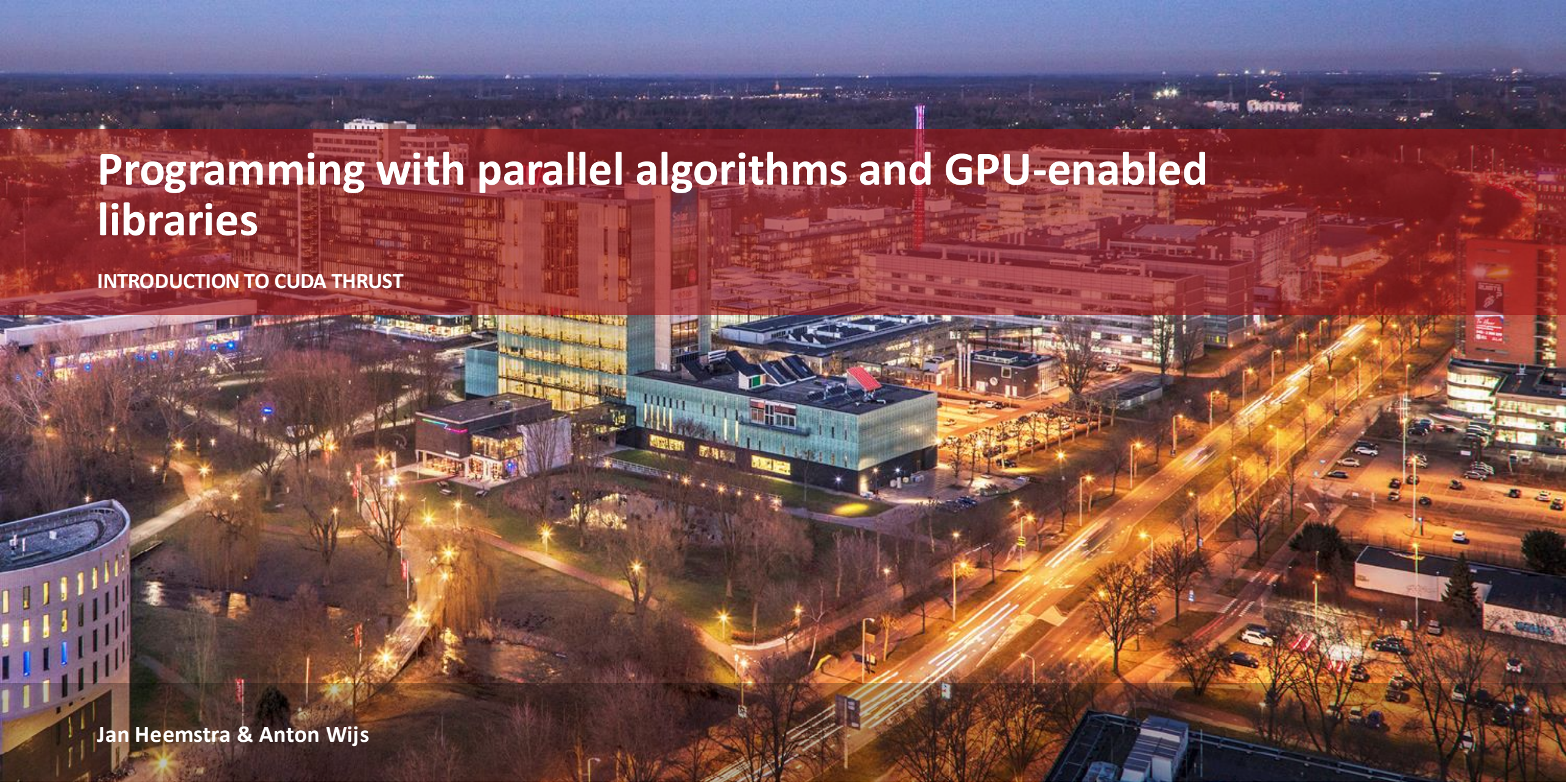


Programming with parallel algorithms and GPU-enabled libraries

INTRODUCTION TO CUDA THRUST

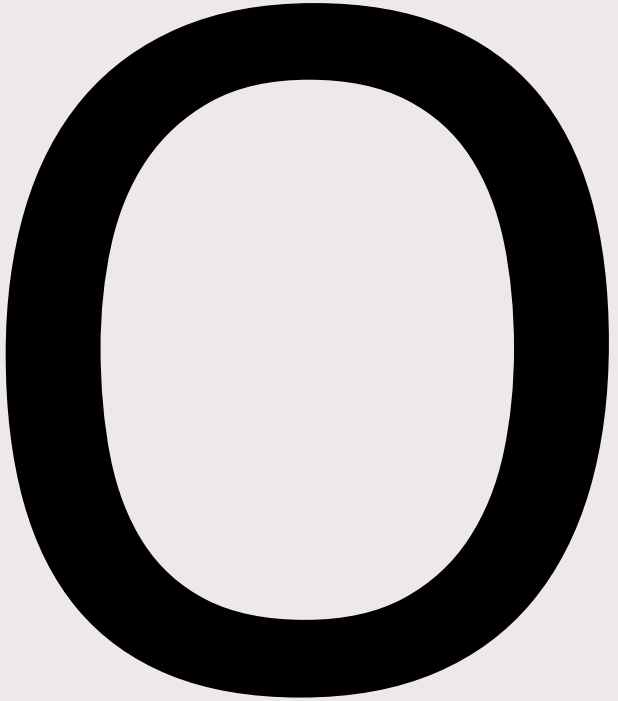
Jan Heemstra & Anton Wijs

STEAM academy 2026

TU/e EINDHOVEN UNIVERSITY OF TECHNOLOGY

Big O notation

- Bound on running time
- g(n) ∈ O(f(n)) means there exists some constants c and d, such that:
  - g(n) < c * f(n)
- holds for all n > d.
- Common (but incorrect) notation:
- g(n) = O(f(n)



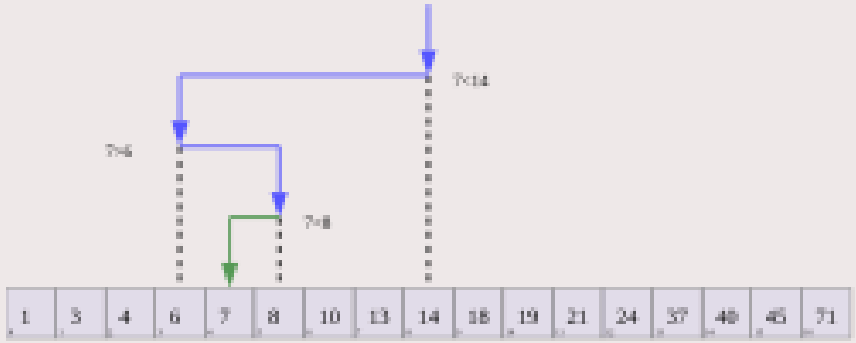
Some examples:

Copying n elements is linear time, or O(n)

Binary search is logarithmic time, or O(log(n))

Bubble sort is quadratic time, or O(n^2)

Best complexity for sorting is O(n log(n))



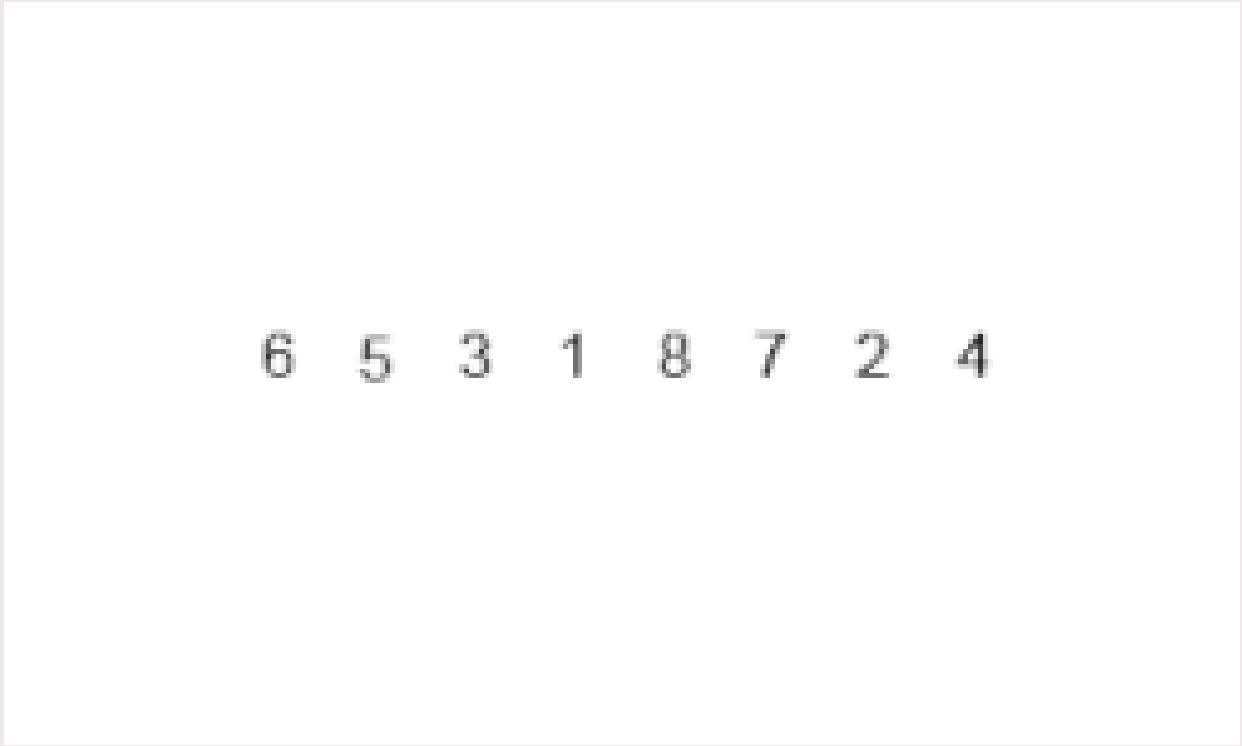
https://en.wikipedia.org/wiki/Binary_search#/media/File:Binary_Search_Depiction.svg

Cache lines

- Memory is delivered to the cache in chunks.
- Multiple levels of memory/cache
  - VRAM
  - Level 2 cache
  - Level 1 cache (Per SM)
  - Registers

Sorting

- O(n log(n)) for sequential compute.
- Bottleneck on GPU is memory throughput.
  - Compute outpaces memory throughput.
- Memory throughput constrained complexity:
  - N: # of elements to sort
  - M: # of records in internal memory
  - B: # records transferred in a cache line transaction
- # of memory transactions is in $O(\frac{N}{B}\ \frac{\log(\frac{N}{B})}{\log(\frac{M}{B})})$



https://en.wikipedia.org/wiki/Merge_sort#/media/File:Merge-sort-example-300px.gif

Thrust

- Algorithms
  - thrust::transform
  - thrust::sort thrust::sort_by_key
  - thrust::lower_bound thrust::upper_bound
  - thrust::exclusive_scan thrust::inclusive_scan
  - thrust::gather thrust::scatter
  - thrust::for_each
  - thrust::unique thrust::remove_if
- Algorithm takes execution policy and iterators as parameters.
  - What is an iterator?

Thrust

- Iterators
  - thrust::make_transform_iterator
  - thrust::make_constant_iterator
  - thrust::make_counting_iterator
  - thrust::make_permutation_iterator
  - thrust::make_zip_iterator
- “Read head” for data.
- Iterators can also be made directly from vectors.

Thrust

- Vectors
  - thrust::host_vector
  - thrust::universal_vector
  - thrust::device_vector
- Begin and end iterators for a vector v can be obtained by:
  - v.begin()
  - v.end()
- Size can be obtained by:
  - v.size()
- This is equal to:
  - v.end() – v.begin()

```
thrust::sort(
  thrust::device,
  v.begin(), v.end()
);
```

Thrust

- Functors
  - cuda::std::plus cuda::std::minus
  - cuda::std::equal_to cuda::std::not_equal_to
- Object that behaves as function
- Overload operator()
- Used in some thrust algorithms, like thrust::transform.

```
struct apply_and{
    __host__ __device__
    uint32_t operator() (uint32_t a, uint32_t b) const {
        return a && b;
    }
};
```

Thrust

- Documentation:
  - https://nvidia.github.io/cccl/unstable/thrust/index.html
- Use ctrl+k to search

Quick note on lambdas

A lambda is a functor that can be defined inline.

[] __host__ __device__ (type1 varname1, type2 varname2) {function body;}

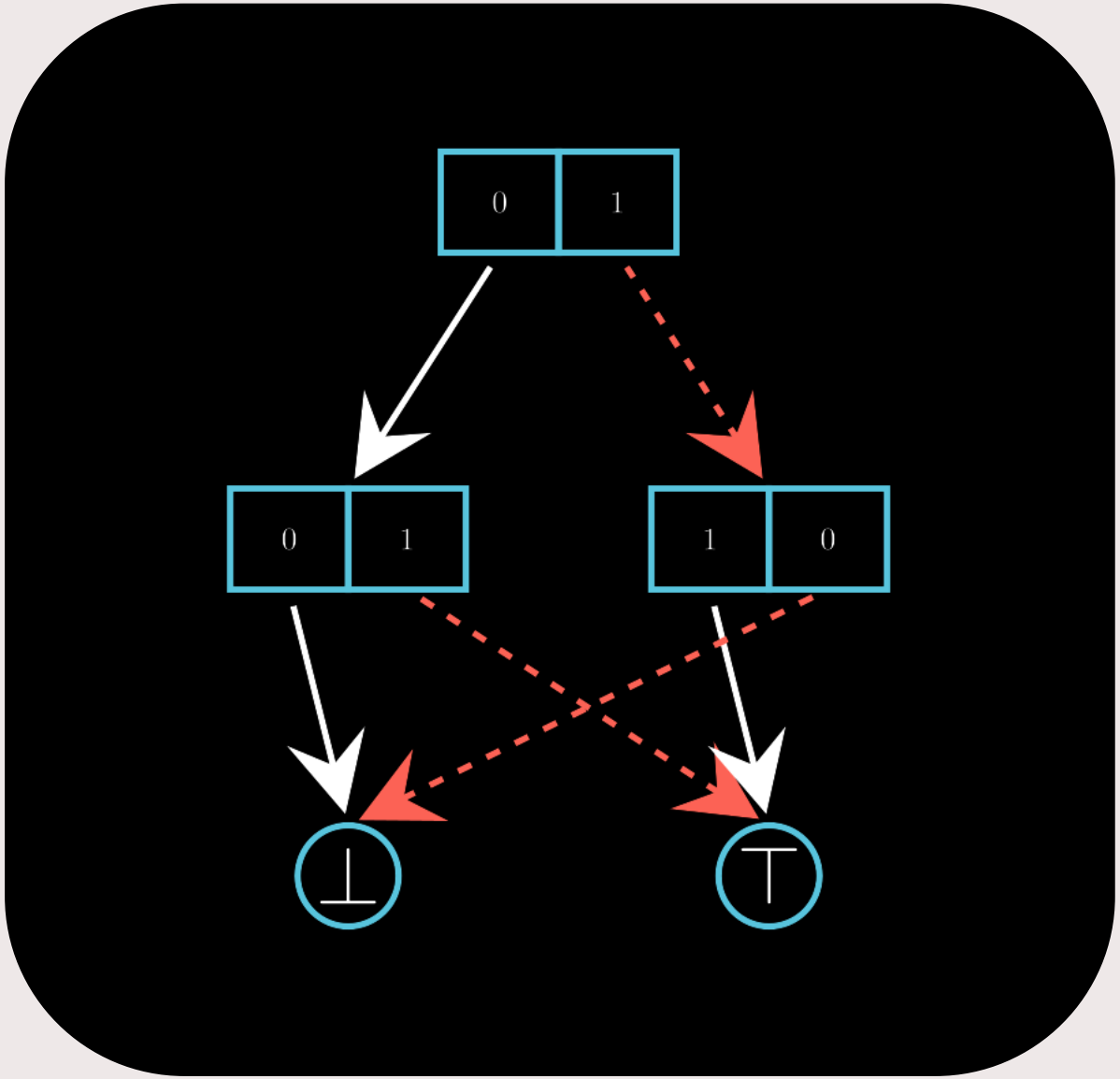
[] -> Scope indicator: keep empty for __device__
(…) -> function parameters
{…} -> function body

Structs and arrays

- Struct of arrays (SoA)
  - Industry standard
  - “Always use this” 🤦‍♀️
- Array of Structs (AoS)
  - Better for sorting
- $O(\frac{N}{B}\ \frac{\log(\frac{N}{B})}{\log(\frac{M}{B})})$

GPUdecide

- GPU-accelerated BDD library
- Very early in development
  - First commit in September 2025
  - First nontrivial BDDs constructed in December 2025



Exercise

- A sabotaged version of GPUdecide is available in exercise 6
- Compilation instructions are in readme.md
  - Create python virtual environment
  - Install scons and pybind11
  - Compile with scons –j32

Exercise

- Implement previously described algorithm
- Compilation instructions are in readme.md
- Put function in
- The n-queens puzzle can be run by executing:
  python examples/n_queens.py –n 8
- Parameter -n controls board size
- GPUdecide works without garbage collection
  - But nodes are leaking!
- One pitfall will be mentioned soon in a hint

Exercise

```
template <typename Policy, typename ChildVec, typename ParentVec>
void garbage_collection(Policy policy, ChildVec& v, ParentVec& parent_v) {
    // Templating logic to deduce type of vector (host/device)
    using Vec = std::remove_reference_t<decltype(v)>;


    // Templating logic to construct the right type of vector (host/device) with our own type (uint32_t)
    // StencilVec is thrust::device_vector<uint32_t> or thrust::host_vector<uint32_t>
    using StencilVec = rebind_vector_t<Vec, uint32_t>; // Why use uint32_t and not bool?

    // Generate list of referenced nodes
    // This initializes a thrust::(host|device)_vector called stencil with v.size() elements, all initialized to `1`
    StencilVec stencil(v.size(), 1);
};
```

Exercise

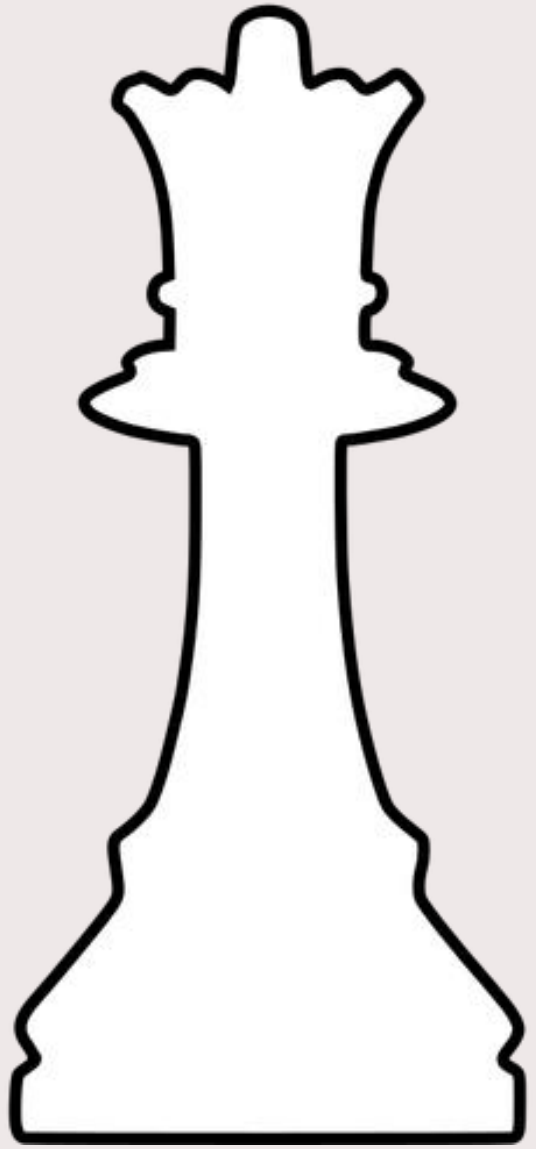
- Hint: what happens in the layer above?

Exercise

- An unsabotaged version of GPUdecide is available in exercise 7
- If you are done with exercise 6, you could use that version.
- Compilation instructions are in readme.md
  - Create python virtual environment
  - Install scons and pybind11
  - Compile with scons –j32

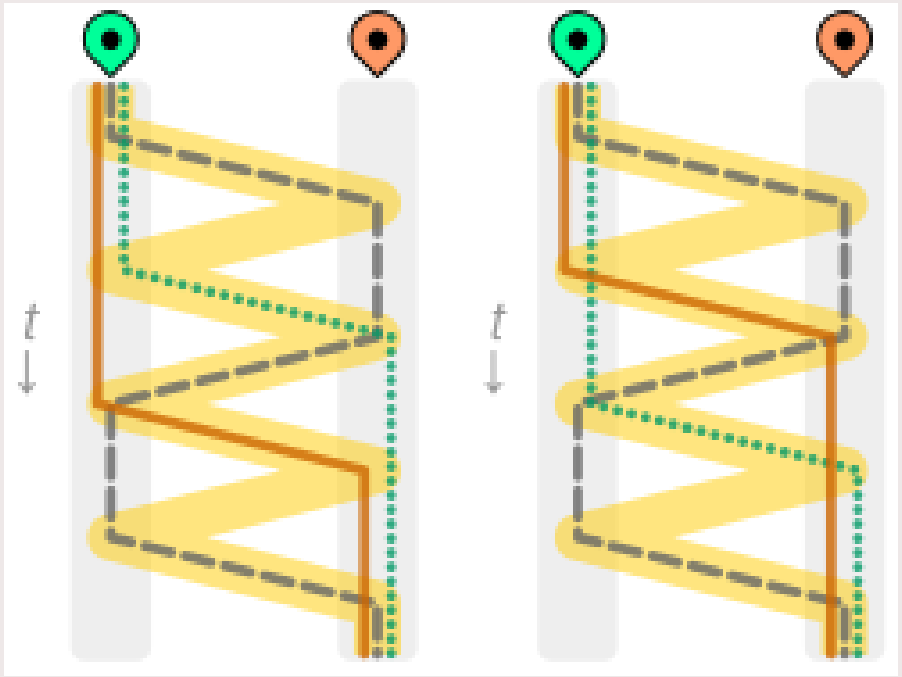
Challenge Exercise

- GPUdecide puzzle solver
- N-queens puzzle has been implemented
- Come up with your own puzzle!
  - Sliding puzzle?
  - Rubiks cube?
  - How many knights fit on the chess board?
  - Your own puzzle?
- Implement with the python bindings
- Have fun!

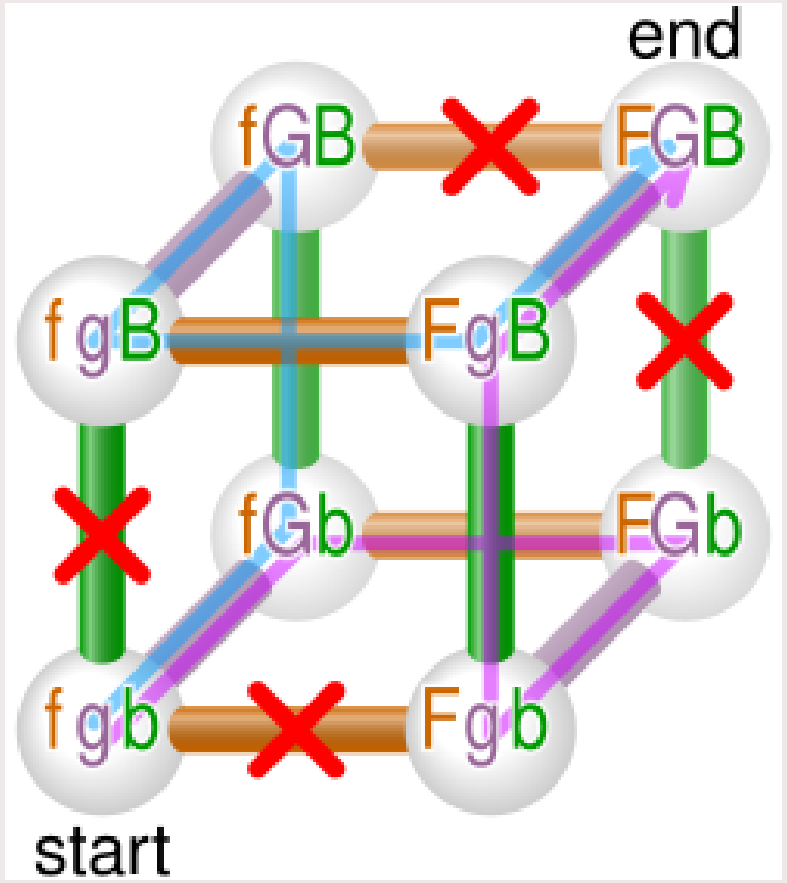


Challenge Exercise

- Simple puzzle:
  - Wolf, goat and cabbage problem



https://en.wikipedia.org/wiki/Wolf,_goat_and_cabbage_problem#/media/File:Fox_goose_beans_puzzle_solution.svg



https://en.wikipedia.org/wiki/Wolf,_goat_and_cabbage_problem#/media/File:Fox_goose_beans_puzzle_visualisation.svg